

好题选讲

雅礼中学 范梓轩

题意 有 n 个格子排成一行，第 i 个格子上有一个左右箭头 C_i 。

有 m 种操作，开始第 i 个操作后，会有一个球出现在 A_i 上，并且 A_i 上的箭头变成朝右。然后球会不断根据它所在的格子上箭头的朝向移动，并在离开一个格子后翻转其上的箭头方向，直到到达 0 或 $n + 1$ ，此时操作结束。

有 q 次独立的询问，每次询问中给出一个区间 $[L_i, R_i]$ ，你需要求出在依次运行第 $L_i, L_i + 1, \dots, R_i$ 种操作后，有多少个朝右的箭头。

$n, m, q \leq 1.2 \times 10^5$ 。部分分：保证 C_i 的一段前缀都是右箭头，后缀都是左箭头。

一次操作中，球的运动形如：向右直到碰到第一个左箭头，再向左直到碰到第一个右箭头。每一次转向后球的运动范围都会包含之前经过的格子，所以我们只在乎它最后一次转向的位置与方向。如果是朝左，则一段前缀最终都会变成右箭头，其他箭头不变，反之亦然。

一次操作中，球的运动形如：向右直到碰到第一个左箭头，再向左直到碰到第一个右箭头。

每一次转向后球的运动范围都会包含之前经过的格子，所以我们只在乎它最后一次转向的位置与方向。如果是朝左，则一段前缀最终都会变成右箭头，其他箭头不变，反之亦然。

定量分析一下。假设球的初始位置是 i ，且其上的箭头已经变成右箭头了。那么一次操作结束后发生的事要么是前 i 个左箭头变成了右箭头，要么是后 $n - i + 1$ 个右箭头变成了左箭头。

一次操作中，球的运动形如：向右直到碰到第一个左箭头，再向左直到碰到第一个右箭头。

每一次转向后球的运动范围都会包含之前经过的格子，所以我们只在乎它最后一次转向的位置与方向。如果是朝左，则一段前缀最终都会变成右箭头，其他箭头不变，反之亦然。

定量分析一下。假设球的初始位置是 i ，且其上的箭头已经变成右箭头了。那么一次操作结束后发生的事要么是前 i 个左箭头变成了右箭头，要么是后 $n - i + 1$ 个右箭头变成了左箭头。

而更进一步地，可以发现这两种操作不可能同时可以进行，所以我们只需要知道当前左箭头的数量，就能判定这次操作是哪一种情况。

来看部分分：保证 C_i 的一段前缀都是右箭头，后缀都是左箭头。

来看部分分：保证 C_i 的一段前缀都是右箭头，后缀都是左箭头。

此时因为每次操作都是反转一段前缀的左箭头或反转一段后缀的右箭头，所以 C_i 经过操作后仍然满足这一性质，我们只需要维护分界点 t 满足 $C[1:t]$ 为右箭头， $C[t+1:n]$ 为左箭头。

来看部分分：保证 C_i 的一段前缀都是右箭头，后缀都是左箭头。

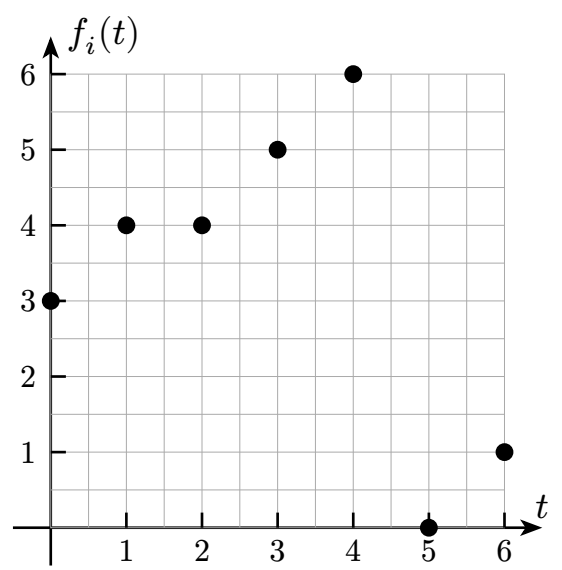
此时因为每次操作都是反转一段前缀的左箭头或反转一段后缀的右箭头，所以 C_i 经过操作后仍然满足这一性质，我们只需要维护分界点 t 满足 $C[1:t]$ 为右箭头， $C[t+1:n]$ 为左箭头。

可以发现一次操作对 t 的影响可以写成一个分段函数

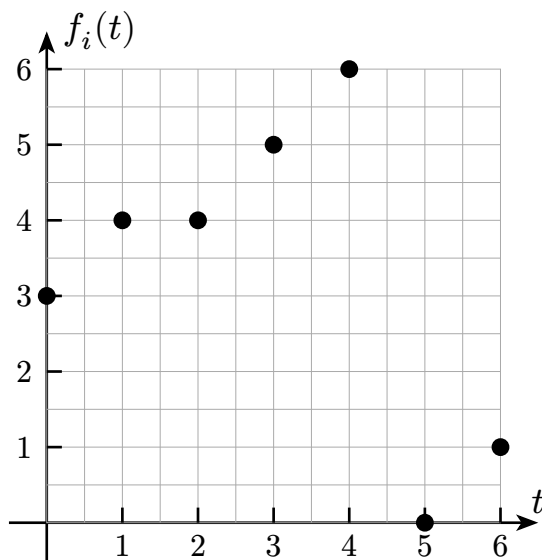
$$f_i(t) = \begin{cases} (t + A_i + 1) \bmod (n + 1) & \text{if } t < A_i \\ (t + A_i) \bmod (n + 1) & \text{if } t \geq A_i \end{cases}$$

而单次询问的答案即为 $(f_{R_i} \circ f_{R_i-1} \circ \cdots \circ f_{L_i})(t)$ 。

考察 $f_i(t)$ 的函数图像 (下图中 $n = 6, A_i = 2$):



考察 $f_i(t)$ 的函数图像（下图中 $n = 6$, $A_i = 2$ ）：



可以观察到 $\{f_i(t) \mid 0 \leq t \leq n\}$ 几乎形成了一个 $[0, \dots, n]$ 的排列，而至多有一个数出现了两次。所以当我们把一个分段函数右复合上 f_i 后，新函数的段数只会比原函数多 $\Theta(1)$ 段。

因此 $(f_{R_i} \circ f_{R_i-1} \circ \cdots \circ f_{L_i})(t)$ 是一个 $\Theta(R_i - L_i)$ 段的分段函数。我们可以直接用线段树维护区间内所有 f_i 复合后的结果，而复合两个 l 段的分段函数可以做到 $\Theta(l \log l)$ 的时间复杂度（这需要复合后的函数分段数量也是 $O(l)$ 的），我们得以 $\Theta(n \log^2 n)$ 地建出这棵线段树。

因此 $(f_{R_i} \circ f_{R_i-1} \circ \cdots \circ f_{L_i})(t)$ 是一个 $\Theta(R_i - L_i)$ 段的分段函数。我们可以直接用线段树维护区间内所有 f_i 复合后的结果，而复合两个 l 段的分段函数可以做到 $\Theta(l \log l)$ 的时间复杂度（这需要复合后的函数分段数量也是 $O(l)$ 的），我们得以 $\Theta(n \log^2 n)$ 地建出这棵线段树。

而查询时，因为求一个 l 段的分段函数在单点处的值可以做到 $\Theta(\log l)$ ，所以查询的时间复杂度为 $\Theta(q \log^2 n)$ ，这就做完了部分分。

回到原题，我们发现若干次操作后的 C' 有形如 $R^l C[l+1:r-1] L^r$ 的结构，而每次操作将会延长 l 或 r ，直到它们相交，此时 C' 满足特殊性质，可以套用部分分做法。所以我们只需要维护 l, r ，并尝试快速处理它们不交的阶段。

回到原题，我们发现若干次操作后的 C' 有形如 $R^l C[l+1:r-1] L^r$ 的结构，而每次操作将会延长 l 或 r ，直到它们相交，此时 C' 满足特殊性质，可以套用部分的做法。所以我们只需要维护 l, r ，并尝试快速处理它们不交的阶段。

如果一次操作满足 $A_i \leq l$ ，则我们可以假设此时进行的操作一定是反转前 A_i 个左括号，这是因为反转后 $n - A_i + 1$ 个右括号一定会波及到前 l 个右括号，所以这种操作只会出现一次。 $A_i \geq n - r + 1$ 时同理。

回到原题，我们发现若干次操作后的 C' 有形如 $R^l C[l+1:r-1] L^r$ 的结构，而每次操作将会延长 l 或 r ，直到它们相交，此时 C' 满足特殊性质，可以套用部分的做法。所以我们只需要维护 l, r ，并尝试快速处理它们不交的阶段。

如果一次操作满足 $A_i \leq l$ ，则我们可以假设此时进行的操作一定是反转前 A_i 个左括号，这是因为反转后 $n - A_i + 1$ 个右括号一定会波及到前 l 个右括号，所以这种操作只会出现一次。 $A_i \geq n - r + 1$ 时同理。

而当 $l < A_i < n - r + 1$ 时，我们就不能假设这次操作的效果了。不过可以发现的是，不管这次操作效果如何， l 与 r 定有一者至少会翻倍。所以这样的情况只会发生 $O(\log n)$ 次。

回到原题，我们发现若干次操作后的 C' 有形如 $R^l C[l+1:r-1] L^r$ 的结构，而每次操作将会延长 l 或 r ，直到它们相交，此时 C' 满足特殊性质，可以套用部分的做法。所以我们只需要维护 l, r ，并尝试快速处理它们不交的阶段。

如果一次操作满足 $A_i \leq l$ ，则我们可以假设此时进行的操作一定是反转前 A_i 个左括号，这是因为反转后 $n - A_i + 1$ 个右括号一定会波及到前 l 个右括号，所以这种操作只会出现一次。 $A_i \geq n - r + 1$ 时同理。

而当 $l < A_i < n - r + 1$ 时，我们就不能假设这次操作的效果了。不过可以发现的是，不管这次操作效果如何， l 与 r 定有一者至少会翻倍。所以这样的情况只会发生 $O(\log n)$ 次。

这给出一个初步的做法：我们找到第一个满足 $l < A_i < n - r + 1$ 的操作，并统计之前的操作中满足 $A_i \leq l$ 的所有 A 的和与满足 $A_i \geq n - r + 1$ 的所有 A 的和。如果之前的操作中 l 和 r 已经相交，我们就二分找到首次相交的时间；否则再暴力处理当前的这次操作即可。

但是 l 和 r 是在动态变化的，所以我们并不好找到第一个满足 $l < A_i < n - r + 1$ 的操作来暴力处理。不过我们可以适当放宽这一限制，也就是找到两个固定点 L, R 满足 $L \leq l$ 且 $R \leq r$ ，则找到第一个满足 $L < A_i < n - R + 1$ 操作来暴力处理并不会影响正确性，我们只需要找到合适的一系列固定点使得时间复杂度仍有保障即可。

但是 l 和 r 是在动态变化的，所以我们并不好找到第一个满足 $l < A_i < n - r + 1$ 的操作来暴力处理。不过我们可以适当放宽这一限制，也就是找到两个固定点 L, R 满足 $L \leq l$ 且 $R \leq r$ ，则找到第一个满足 $L < A_i < n - R + 1$ 操作来暴力处理并不会影响正确性，我们只需要找到合适的一系列固定点使得时间复杂度仍有保障即可。

recall 一下我们之前发现的性质：一次暴力操作定会使得 l 或 r 翻倍。这启示我们取 $L = 2^0, 2^1, \dots, R = 2^0, 2^1, \dots$ ，这样一来，每次暴力操作都会使得 L 或 R 改变。对每一对 L, R 预处理出相关信息即可，时间复杂度 $\Theta((n + q) \log^2 n)$ 。

题意 给定正整数 N, M 与 M 个约束 (A_i, B_i) 。你需要计数：有多少不同的有标号的以 1 为根的 N 个点的二叉树，使得其先序遍历恰为 $[1, 2, \dots, N]$ ，中序遍历的第 B_i 项恰为点 A_i 。

$M \leq N \leq 500$ 。

qoj17247 Binary Tree Counting

直接在二叉树上计数并不方便，考虑先把它和一些别的组合结构建立双射。

qoj17247 Binary Tree Counting

直接在二叉树上计数并不方便，考虑先把它和一些别的组合结构建立双射。

考虑括号串，设根的左右子树分别为 T_1, T_2 ，我们将这棵二叉树映射到 $(T_1)T_2$ 。显然这是一个双射，并且其先序遍历的第 i 项就是第 i 个左括号对应的节点，中序遍历的第 i 项就是第 i 个右括号对应的节点。则限制改写为第 A_i 个左括号要和第 B_i 个右括号匹配。

qoj17247 Binary Tree Counting

直接在二叉树上计数并不方便，考虑先把它和一些别的组合结构建立双射。

考虑括号串，设根的左右子树分别为 T_1, T_2 ，我们将这棵二叉树映射到 $(T_1)T_2$ 。显然这是一个双射，并且其先序遍历的第 i 项就是第 i 个左括号对应的节点，中序遍历的第 i 项就是第 i 个右括号对应的节点。则限制改写为第 A_i 个左括号要和第 B_i 个右括号匹配。

还是不好计数，再做一步转化，把它映射到 $(0, 0)$ 到 (N, N) 的格路上，左括号对应向右，右括号对应向上。

qoj17247 Binary Tree Counting

直接在二叉树上计数并不方便，考虑先把它和一些别的组合结构建立双射。

考虑括号串，设根的左右子树分别为 T_1, T_2 ，我们将这棵二叉树映射到 $(T_1)T_2$ 。显然这是一个双射，并且其先序遍历的第 i 项就是第 i 个左括号对应的节点，中序遍历的第 i 项就是第 i 个右括号对应的节点。则限制改写为第 A_i 个左括号要和第 B_i 个右括号匹配。

还是不好计数，再做一步转化，把它映射到 $(0, 0)$ 到 (N, N) 的格路上，左括号对应向右，右括号对应向上。

那么对于一个限制 (A_i, B_i) ，我们就知道第 A_i 个左括号对应的边一定形如一条 $(A_i - 1, x)$ 出发向右的边，第 B_i 个右括号对应的边一定形如一条 $(y, B_i - 1)$ 出发向上的边。

qoj17247 Binary Tree Counting

直接在二叉树上计数并不方便，考虑先把它和一些别的组合结构建立双射。

考虑括号串，设根的左右子树分别为 T_1, T_2 ，我们将这棵二叉树映射到 $(T_1)T_2$ 。显然这是一个双射，并且其先序遍历的第 i 项就是第 i 个左括号对应的节点，中序遍历的第 i 项就是第 i 个右括号对应的节点。则限制改写为第 A_i 个左括号要和第 B_i 个右括号匹配。

还是不好计数，再做一步转化，把它映射到 $(0, 0)$ 到 (N, N) 的格路上，左括号对应向右，右括号对应向上。

那么对于一个限制 (A_i, B_i) ，我们就知道第 A_i 个左括号对应的边一定形如一条 $(A_i - 1, x)$ 出发向右的边，第 B_i 个右括号对应的边一定形如一条 $(y, B_i - 1)$ 出发向上的边。又因为这两个括号需要匹配，所以如果我们还知道它们之间的括号数量为 $2l$ ，就能唯一确定 $x = B_i - 1 - l$ 、 $y = A_i + l$ 。

qoj17247 Binary Tree Counting

再看限制之间的关系，对于两个限制 (A_i, B_i) 、 (A_j, B_j) ($A_i < A_j$)，如果 $B_i < B_j$ ，则这两个限制必然不交；否则 i 必然包含 j 。于是限制之间形成了一棵树形结构，我们就能在这棵树上 dp 了。

qoj17247 Binary Tree Counting

再看限制之间的关系，对于两个限制 (A_i, B_i) 、 (A_j, B_j) ($A_i < A_j$)，如果 $B_i < B_j$ ，则这两个限制必然不交；否则 i 必然包含 j 。于是限制之间形成了一棵树形结构，我们就能在这棵树上 dp 了。

设 $f_{i,l}$ 表示只考虑第 i 个限制及其子树内的限制，要求第 A_i 个左括号和第 B_i 个右括号之间恰有 $2l$ 个括号时的方案数。

qoj17247 Binary Tree Counting

再看限制之间的关系，对于两个限制 (A_i, B_i) 、 (A_j, B_j) ($A_i < A_j$)，如果 $B_i < B_j$ ，则这两个限制必然不交；否则 i 必然包含 j 。于是限制之间形成了一棵树形结构，我们就能在这棵树上 dp 了。

设 $f_{i,l}$ 表示只考虑第 i 个限制及其子树内的限制，要求第 A_i 个左括号和第 B_i 个右括号之间恰有 $2l$ 个括号时的方案数。

考虑单个 $f_{i,l}$ 的转移。我们依次加入 i 的所有儿子 $u_1, \dots, u_k$ ，则加入一个儿子时我们只需要枚举上一个儿子用掉的 l 与当前儿子用掉的 l 就能计算出中间的转移系数。

qoj17247 Binary Tree Counting

再看限制之间的关系，对于两个限制 (A_i, B_i) 、 (A_j, B_j) ($A_i < A_j$)，如果 $B_i < B_j$ ，则这两个限制必然不交；否则 i 必然包含 j 。于是限制之间形成了一棵树形结构，我们就能在这棵树上 dp 了。

设 $f_{i,l}$ 表示只考虑第 i 个限制及其子树内的限制，要求第 A_i 个左括号和第 B_i 个右括号之间恰有 $2l$ 个括号时的方案数。

考虑单个 $f_{i,l}$ 的转移。我们依次加入 i 的所有儿子 $u_1, \dots, u_k$ ，则加入一个儿子时我们只需要枚举上一个儿子用掉的 l 与当前儿子用掉的 l 就能计算出中间的转移系数。

这个做法看上去是 $\Theta(MN^3)$ 的，但是仔细分析一下，可以发现转移的时间复杂度其实是 $\Theta\left(\left(A_{u_{j+1}} - A_{u_j}\right)\left(B_{u_{j+1}} - B_{u_j}\right)\right)$ 的。

qoj17247 Binary Tree Counting

再看限制之间的关系，对于两个限制 (A_i, B_i) 、 (A_j, B_j) ($A_i < A_j$)，如果 $B_i < B_j$ ，则这两个限制必然不交；否则 i 必然包含 j 。于是限制之间形成了一棵树形结构，我们就能在这棵树上 dp 了。

设 $f_{i,l}$ 表示只考虑第 i 个限制及其子树内的限制，要求第 A_i 个左括号和第 B_i 个右括号之间恰有 $2l$ 个括号时的方案数。

考虑单个 $f_{i,l}$ 的转移。我们依次加入 i 的所有儿子 $u_1, \dots, u_k$ ，则加入一个儿子时我们只需要枚举上一个儿子用掉的 l 与当前儿子用掉的 l 就能计算出中间的转移系数。

这个做法看上去是 $\Theta(MN^3)$ 的，但是仔细分析一下，可以发现转移的时间复杂度其实是 $\Theta\left(\left(A_{u_{j+1}} - A_{u_j}\right)\left(B_{u_{j+1}} - B_{u_j}\right)\right)$ 的。将 (A_i, B_i) 看作平面上的点，则不难发现这些矩形两两不交，所以总时间复杂度实为 $\Theta((M + N)N^2)$ 。

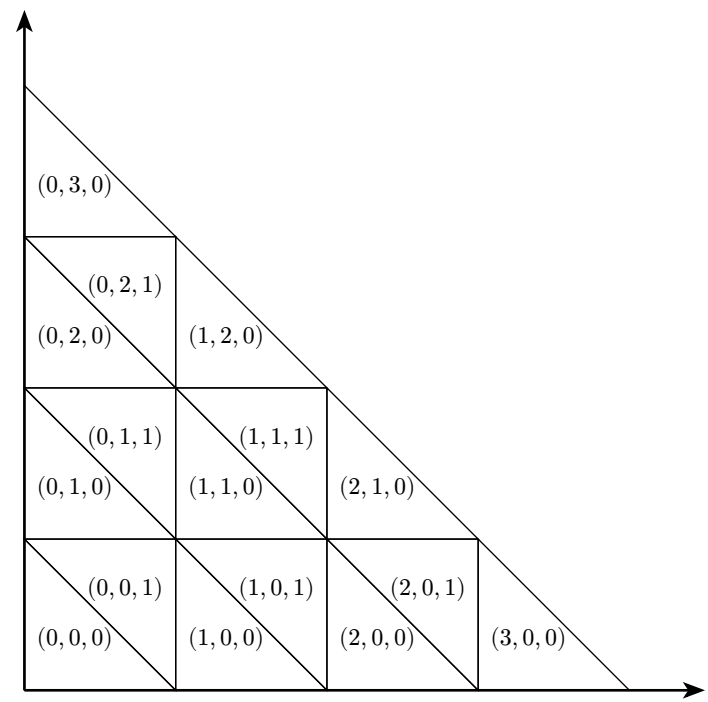
P15945 [JOI 2026 Final] 雨落三角 / Triangular Rainfall

题意 在二维平面上，定义三角形 $(x, y, 0)$ 为 $(x, y) - (x + 1, y) - (x, y + 1)$ 围成的区域，三角形 $(x, y, 1)$ 为 $(x + 1, y) - (x, y + 1) - (x + 1, y + 1)$ 围成的区域。

有 N 个大三角形，第 i 个三角形为 $(X_i, Y_i) - (X_i + Z_i, Y_i) - (X_i, Y_i + Z_i)$ 围成的区域。你需要对所有 $k \in [0, K - 1]$ ，输出有多少个满足 $x + y + z < L$ 且被恰好 k 个大三角形覆盖的三角形 (x, y, z) 。

$N \leq 2 \times 10^5$, $L \leq 10^9$, $K \leq 5$ 。

P15945 [JOI 2026 Final] 雨落三角 / Triangular Rainfall



P15945 [JOI 2026 Final] 雨落三角 / Triangular Rainfall

$z = 0$ 的三角形和 $z = 1$ 的三角形显然可以分开维护，接下来只看 $z = 0$ 的那些三角形。

P15945 [JOI 2026 Final] 雨落三角 / Triangular Rainfall

$z = 0$ 的三角形和 $z = 1$ 的三角形显然可以分开维护，接下来只看 $z = 0$ 的那些三角形。

一个区域 $(X, Y) - (X + Z, Y) - (X, Y + Z)$ 会覆盖三角形 (x, y) 当且仅当 $X \leq x \wedge Y \leq y \wedge x + y < X + Y + Z$ 。下面把一个区域记作 (X, Y, T) 表示其覆盖了满足 $X \leq x \wedge Y \leq y \wedge x + y \leq T$ 的所有三角形 (x, y) 。

P15945 [JOI 2026 Final] 雨落三角 / Triangular Rainfall

$z = 0$ 的三角形和 $z = 1$ 的三角形显然可以分开维护，接下来只看 $z = 0$ 的那些三角形。

一个区域 $(X, Y) - (X + Z, Y) - (X, Y + Z)$ 会覆盖三角形 (x, y) 当且仅当 $X \leq x \wedge Y \leq y \wedge x + y < X + Y + Z$ 。下面把一个区域记作 (X, Y, T) 表示其覆盖了满足 $X \leq x \wedge Y \leq y \wedge x + y \leq T$ 的所有三角形 (x, y) 。

先看 $K = 1$ 的情况，此时的问题类似于求三角形面积并。

P15945 [JOI 2026 Final] 雨落三角 / Triangular Rainfall

$z = 0$ 的三角形和 $z = 1$ 的三角形显然可以分开维护，接下来只看 $z = 0$ 的那些三角形。

一个区域 $(X, Y) - (X + Z, Y) - (X, Y + Z)$ 会覆盖三角形 (x, y) 当且仅当 $X \leq x \wedge Y \leq y \wedge x + y < X + Y + Z$ 。下面把一个区域记作 (X, Y, T) 表示其覆盖了满足 $X \leq x \wedge Y \leq y \wedge x + y \leq T$ 的所有三角形 (x, y) 。

先看 $K = 1$ 的情况，此时的问题类似于求三角形面积并。仍然考虑扫描线，在扫描到 x_0 的时候加入所有满足 $X = x_0$ 的区域。则 y 一维上的限制分为 $Y \leq y$ 和 $x + y \leq T$ 两种，我们可以在每个点 (x, y) 上记录 a_y 表示限制最宽松的那个 T ，那么加入一个区域时就只需要对 a 的一段后缀 `chkmax`。

P15945 [JOI 2026 Final] 雨落三角 / Triangular Rainfall

$z = 0$ 的三角形和 $z = 1$ 的三角形显然可以分开维护，接下来只看 $z = 0$ 的那些三角形。

一个区域 $(X, Y) - (X + Z, Y) - (X, Y + Z)$ 会覆盖三角形 (x, y) 当且仅当 $X \leq x \wedge Y \leq y \wedge x + y < X + Y + Z$ 。下面把一个区域记作 (X, Y, T) 表示其覆盖了满足 $X \leq x \wedge Y \leq y \wedge x + y \leq T$ 的所有三角形 (x, y) 。

先看 $K = 1$ 的情况，此时的问题类似于求三角形面积并。仍然考虑扫描线，在扫描到 x_0 的时候加入所有满足 $X = x_0$ 的区域。则 y 一维上的限制分为 $Y \leq y$ 和 $x + y \leq T$ 两种，我们可以在每个点 (x, y) 上记录 a_y 表示限制最宽松的那个 T ，那么加入一个区域时就只需要对 a 的一段后缀 `chkmax`。

因为只有后缀 `chkmax`，所以 a 单调不减，后缀 `chkmax` 实为区间推平，可以颜色段均摊。那么我们可以直接维护每种 $a_y - y$ 的出现次数，查询时只需要算一个区间和。

P15945 [JOI 2026 Final] 雨落三角 / Triangular Rainfall

$z = 0$ 的三角形和 $z = 1$ 的三角形显然可以分开维护，接下来只看 $z = 0$ 的那些三角形。

一个区域 $(X, Y) - (X + Z, Y) - (X, Y + Z)$ 会覆盖三角形 (x, y) 当且仅当 $X \leq x \wedge Y \leq y \wedge x + y < X + Y + Z$ 。下面把一个区域记作 (X, Y, T) 表示其覆盖了满足 $X \leq x \wedge Y \leq y \wedge x + y \leq T$ 的所有三角形 (x, y) 。

先看 $K = 1$ 的情况，此时的问题类似于求三角形面积并。仍然考虑扫描线，在扫描到 x_0 的时候加入所有满足 $X = x_0$ 的区域。则 y 一维上的限制分为 $Y \leq y$ 和 $x + y \leq T$ 两种，我们可以在每个点 (x, y) 上记录 a_y 表示限制最宽松的那个 T ，那么加入一个区域时就只需要对 a 的一段后缀 `chkmax`。

因为只有后缀 `chkmax`，所以 a 单调不减，后缀 `chkmax` 实为区间推平，可以颜色段均摊。那么我们可以直接维护每种 $a_y - y$ 的出现次数，查询时只需要算一个区间和。

至于 $K > 1$ 的情况也是一样的，只需要维护前 K 宽松的 T ，并用 K 个数据结构维护即可。一次插入会新增 K 个颜色段，一个颜色段会下放 K 次，故总时间复杂度 $\Theta(NK^2 \log N)$ 。